



Mysten Walrus

Security Assessment

March 3rd, 2025 — Prepared by OtterSec

Sangsoo Kang

sangsoo@osec.io

Michał Bochnak

embe221ed@osec.io

Nicholas R. Putra

nicholas@osec.io

Robert Chen

r@osec.io

Table of Contents

Executive Summary	3
Overview	3
Key Findings	3
Scope	4
Findings	5
Vulnerabilities	6
OS-MSW-ADV-00 Missing Activation Epoch Check in Join	8
OS-MSW-ADV-01 Inconsistencies Due to Zero Share Amount Value	9
OS-MSW-ADV-02 Abort via Large Node Capacity Value	10
OS-MSW-ADV-03 Epoch Mismatch in Storage Reclamation	11
OS-MSW-ADV-04 Division by Zero in Committee Selection	12
OS-MSW-ADV-05 Exceeding Object Size Limit	13
OS-MSW-ADV-06 Missing Commission Rate Check	14
OS-MSW-ADV-07 Missing Check for Sequence Number	15
OS-MSW-ADV-08 Utilization of Incorrect Commission Rate	16
OS-MSW-ADV-09 Bypassing Stake Threshold Check	17
General Findings	18
OS-MSW-SUG-00 Failure to Replace Inactive Node	19
OS-MSW-SUG-01 Missing Validation Logic	20
OS-MSW-SUG-02 Code Refactoring	22
OS-MSW-SUG-03 Code Maturity	23

Appendices

Vulnerability Rating Scale	25
Procedure	26

01 — Executive Summary

Overview

Mysten Labs engaged OtterSec to assess the **walrus-contracts** program. This assessment was conducted between January 21st and February 25th, 2025. For more information on our auditing methodology, refer to [Appendix B](#).

Key Findings

We produced 14 findings during this audit engagement, all of which were successfully remediated in accordance with our recommendations.

In particular, we identified several vulnerabilities. One involves the lack of an activation epoch check when joining the StakedWal, which allows users to inflate their rewards([OS-MSW-ADV-00](#)). Another vulnerability occurs when the share amount is manipulated to be zero in the exchange rate, preventing epoch changes and blocking StakedWal withdrawals([OS-MSW-ADV-01](#)). Additionally, we discovered multiple vulnerabilities that can disrupt epoch advancement, including division by zero, overflow, and object size limits.

We also provided suggestions to address inconsistencies in the codebase and ensure adherence to coding best practices ([OS-MSW-SUG-03](#)) and made recommendations regarding modifications to the codebase for improved functionality, efficiency, and robustness ([OS-MSW-SUG-02](#)). Moreover, we advised including additional safety checks within the codebase to improve security ([OS-MSW-SUG-01](#)).

02 — Scope

The source code was delivered to us in a Git repository at <https://github.com/MystenLabs/walrus>. This audit was performed against commit [a45e012](#).

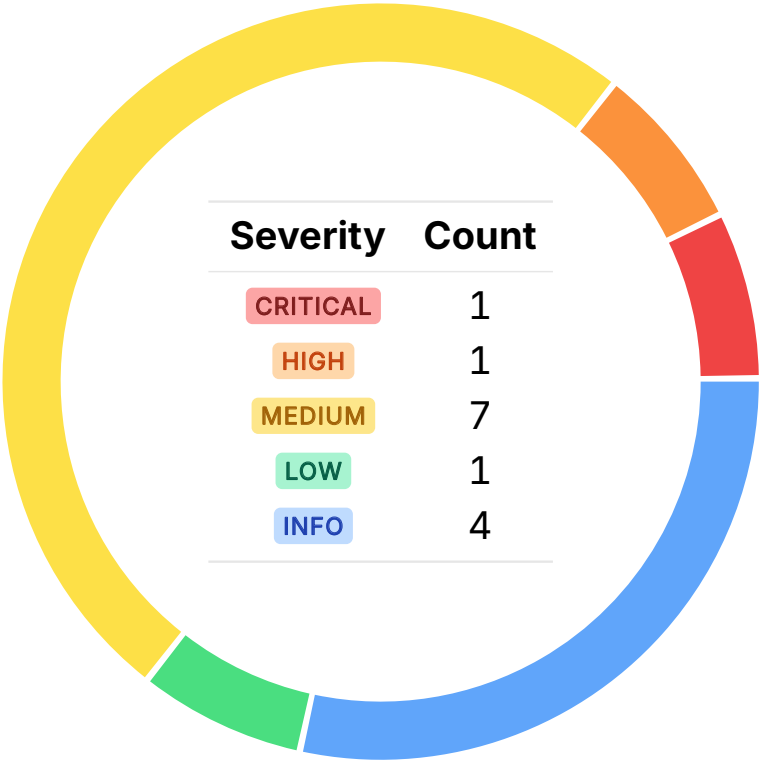
A brief description of the program is as follows:

Name	Description
walrus - contracts	Walrus uses Sui smart contracts to manage storage, payments, and governance, storing only metadata on-chain while keeping blob data off-chain. Users purchase storage, assign a blob ID, and obtain availability certificates, allowing storage nodes to handle and extend storage as needed. At each storage epoch, storage nodes verify performance and distribute payments, ensuring efficient and reliable decentralized storage.

03 — Findings

Overall, we reported 14 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in [Appendix A](#).

ID	Severity	Status	Description
OS-MSW-ADV-00	CRITICAL	RESOLVED ✓	Missing the activation epoch check when joining a <code>StakedWal</code> in the withdrawal state leads to incorrect reward calculations.
OS-MSW-ADV-01	HIGH	RESOLVED ✓	It is possible to manipulate the pool's share-to-asset ratio, as <code>request_withdraw_stake</code> allows zero-share withdrawals currently, and a denial of service scenario may occur due to division by zero in <code>convert_to_wal_amount</code> if share amount is zero.
OS-MSW-ADV-02	MEDIUM	RESOLVED ✓	A malicious user may inflate the node's capacity by setting an excessively large value for <code>node_capacity</code> , disrupting vote calculations and potentially resulting in epoch advancement failures.
OS-MSW-ADV-03	MEDIUM	RESOLVED ✓	Splitting storage by epoch creates a new storage object, but the accounting system may incorrectly decrease storage reclamation for the wrong epoch.
OS-MSW-ADV-04	MEDIUM	RESOLVED ✓	<code>select_committee_and_calculate_votes</code> risks division by zero when calculating <code>capacity_vote</code> if a node's assigned shard count (<code>weight</code>) is zero due to insufficient stake.

OS-MSW-ADV-05	MEDIUM	RESOLVED ✓	Excessive length in <code>StorageNodeInfo.{name,network_address}</code> may inflate the <code>StakingPool</code> size, surpassing the <code>max_move_object_size</code> limit and blocking staker withdrawals by restricting updates to <code>pending_shares_withdraw</code> .
OS-MSW-ADV-06	MEDIUM	RESOLVED ✓	Failing to check whether the commission rate is within the valid range can cause an <code>ENotEnough</code> error, allowing a single node to disrupt the epoch change.
OS-MSW-ADV-07	MEDIUM	RESOLVED ✓	The lack of validation for <code>ending_checkpoint_sequence_num</code> in <code>certify_event_blob</code> can cause mismatches between node-processed event blobs and on-chain tracking.
OS-MSW-ADV-08	MEDIUM	RESOLVED ✓	<code>advance_epoch</code> incorrectly utilizes the updated commission rate for the current epoch instead of the previous one.
OS-MSW-ADV-09	LOW	RESOLVED ✓	<code>ActiveSet::update</code> lacks a threshold check, allowing users to stake above the threshold to enter the active set and then withdraw early to reduce their stake below the minimum requirement while still remaining in the set.

Missing Activation Epoch Check in Join CRITICAL

OS-MSW-ADV-00

Description

When joining `StakedWal`, it must have the same `node_id`, `activation_epoch`, and `state`. However, for a `StakedWal` in the withdrawal state, only the `withdraw_epoch` is checked, while the activation epoch check is missing. Since the share amount is calculated based on the activation epoch, this omission results in incorrect reward calculations, allowing an attacker to gain additional benefits.

```
>_ sources/staking/staked_wal.move
```

RUST

```
public fun join(sw: &mut StakedWal, other: StakedWal) {
    assert!(sw.node_id == other.node_id, EMetadataMismatch);
    [...]

    // Withdrawing scenario - we no longer check that the activation epoch is
    // the same, as the staked WAL is in the process of withdrawing. Instead,
    // we make sure that the withdraw epoch is the same.
    assert!(sw.is_withdrawing() && other.is_withdrawing(), EMetadataMismatch);
    assert!(sw.withdraw_epoch() == other.withdraw_epoch(), EMetadataMismatch);

    let StakedWal { id, principal, .. } = other;
    sw.principal.join(principal);
    id.delete();
}
```

Remediation

Ensure that the activation epoch is the same when joining a `StakedWal` in the withdrawal state.

Patch

Fixed in [3396941](#).

Inconsistencies Due to Zero Share Amount Value HIGH

OS-MSW-ADV-01

Description

`staking_inner::request_withdraw_stake` does not explicitly prevent a withdrawal request with a `share_amount` of zero. This oversight may allow a malicious user to manipulate the staking pool's share-to-asset ratio by withdrawing a small principal or leaving it.

Furthermore, there is a possibility of a denial-of-service attack when `share_amount` is zero in `exchange_rate::convert_to_wal_amount`, due to division by zero when the function performs a division by `share_amount` to compute the `WAL` equivalent. Specifically, this affects epoch advancement and stake withdrawals.

```
>_ sources/staking/exchange_rate.move
```

RUST

```
public(package) fun convert_to_wal_amount(exchange_rate: &PoolExchangeRate, amount: u64): u64 {
  match (exchange_rate) {
    PoolExchangeRate::Flat => amount,
    PoolExchangeRate::Variable { wal_amount, share_amount } => {
      let amount = (amount as u128);
      let res = (amount * *wal_amount) / *share_amount;
      res as u64
    },
  }
}
```

When calculating the pool's active balance, `wal_balance_at_epoch` calls `convert_to_wal_amount`, affecting epoch advancements if `share_amount` is zero. Similarly, during stake withdrawal, `calculate_rewards` internally calls `convert_to_wal_amount` to compute rewards, causing the withdrawal attempt to fail if `share_amount` is zero.

Remediation

Ensure that the minimum amount remains in the pool or `StakedWal`.

Patch

Fixed in [cc4aaaa](#).

Abort via Large Node Capacity Value MEDIUM

OS-MSW-ADV-02

Description

There is a potential flaw in how the vote calculations are handled in `select_committee_and_calculate_votes`, particularly with the `node_capacity` parameter, which may result in an abort. The `node_capacity` refers to the node's capacity in terms of how much load it can handle. If a malicious node sets `node_capacity` to an excessively large value, the capacity vote calculation will result in an extremely high `capacity_vote`.

```
>_ sources/staking/staking_inner.move
```

RUST

```
/// Selects the committee for the next epoch.
public(package) fun select_committee_and_calculate_votes(self: &mut StakingInnerV1) {
    [...]
    // Iterate over the next committee to do the following:
    // - store the next epoch public keys for the nodes
    // - calculate the votes for the next epoch parameters
    node_ids.length().do!(|idx| {
        [...]
        // Perform calculation of the votes.
        write_prices.insert(pool.write_price(), weight);
        storage_prices.insert(pool.storage_price(), weight);
        let capacity_vote = (pool.node_capacity() * (self.n_shards as u64)) / weight;
        capacity_votes.insert(capacity_vote, weight);
    });
    [...]
}
```

`capacity_vote` is calculated as product of `node_capacity` and `n_shards`, divided by `weight`. Thus, if `node_capacity` is excessively large, the result of this multiplication will yield an overflow.

Remediation

Ensure to place a cap on the `node_capacity` value and cast `u64` to `u128`.

Patch

Fixed in [cc4aaaa](#).

Epoch Mismatch in Storage Reclamation

MEDIUM

OS-MSW-ADV-03

Description

`storage_resource::split_by_epoch` modifies an existing storage object by splitting it into two separate objects. The original storage is modified to have `start_epoch → split_epoch`, while a new storage object is created to cover `split_epoch → end_epoch`. However, the accounting mechanism for storage reclamation does not properly adjust to this split, resulting in inconsistencies when `decrease_storage_to_reclaim` is called. Thus, `extend_blob` will not work because `decrease_storage_to_reclaim` in `storage_accounting` will try to decrease the storage size in the wrong epoch.

Remediation

Ensure that storage reclamation is properly tracked for the new epoch by modifying `FutureAccounting` to store `used_capacity` for future epochs.

Patch

Fixed in [f9a7865](#).

Division by Zero in Committee Selection

MEDIUM

OS-MSW-ADV-04

Description

In `staking_inner::select_committee_and_calculate_votes`, a division by zero may occur when calculating `capacity_vote` if `weight` is zero, as the calculation of `capacity_vote` divides the product of `node_capacity` and the number of shards with `weight`. `weight` represents the number of shards assigned to a node in the next epoch. If a node's staked amount is too low, it may not receive any shard assignments, resulting in `shard_assignments[idx].length` to return zero and preventing the voting mechanism from progressing.

```
>_ sources/staking/staking_inner.move
```

RUST

```
/// Selects the committee for the next epoch.
public(package) fun select_committee_and_calculate_votes(self: &mut StakingInnerV1) {
    [...]
    node_ids.length().do!(<|idx| {
        [...]
        // perform calculation of the votes
        write_prices.insert(pool.write_price(), weight);
        storage_prices.insert(pool.storage_price(), weight);
        let capacity_vote = (pool.node_capacity() * (self.n_shards as u64)) / weight;
        capacity_votes.insert(capacity_vote, weight);
    });
    [,,,]
    self.next_epoch_params = option::some(epoch_params);
}
```

Remediation

Skip the `capacity_vote` calculation if the `weight` is 0.

Patch

Fixed in [a1e81c6](#).

Exceeding Object Size Limit MEDIUM

OS-MSW-ADV-05

Description

The issue revolves around the `max_move_object_size` limit, which restricts the maximum size of objects stored on-chain. In `staking_pool`, if the `StorageNodeInfo.{name,network_address}` field is set to an excessively long value by the node owner, it may contribute significantly to the total size of the `StakingPool` object. This may restrict the addition of new values to `pending_shares_withdraw`, thereby blocking stakers from making withdrawals.

Remediation

Enforce a length limit on `StorageNodeInfo.{name,network_address}` to ensure that the overall size of the `StakingPool` object remains within safe bounds.

Patch

Fixed in [a1e81c6](#).

Missing Commission Rate Check MEDIUM

OS-MSW-ADV-06

Description

When creating a node in `staking::register_candidate`, the initial value of `commission_rate` is set. However, since there is no check to ensure that the commission rate is less than or equal to 10000, an `ENotEnough` error may occur in `staking_pool::advance_epoch` when the node becomes part of the committee. As a result, a single node can disrupt the epoch change.

```
>_ sources/staking/staking_pool.move
```

RUST

```
/// Advance epoch for the `StakingPool`.
public(package) fun advance_epoch(
    pool: &mut StakingPool,
    mut rewards: Balance<WAL>,
    wctx: &WalrusContext,
) {
    [...]

    // Split the commission from the rewards.
    let total_rewards = rewards.value();
    let commission = rewards.split(
        total_rewards * (pool.commission_rate as u64) / (N_BASIS_POINTS as u64),
    );
    pool.commission.join(commission);
}
```

Remediation

Ensure that a commission rate doesn't exceed 10000 in creating a node.

Patch

Fixed in [a1e81c6](#).

Missing Check for Sequence Number MEDIUM

OS-MSW-ADV-07

Description

When a node calls `certify_event_blob`, it is required to input the `ending_checkpoint_sequence_num` of the event blob being recorded. While there is validation for the blob ID, there is no validation for the ending checkpoint sequence number. As a result, the value provided by a quorum-reaching node gets recorded in the state. If an incorrect number is entered, it causes a mismatch between the event blob being processed by the node and the on-chain tracking.

Remediation

Track both the blob ID and ending checkpoint sequence number when recording the event blob certification state.

Patch

Fixed in [c873816](#).

Utilization of Incorrect Commission Rate MEDIUM

OS-MSW-ADV-08

Description

The current implementation of `staking_pool::advance_epoch` utilizes the new commission rate when calculating the commission during the epoch advancement process. `advance_epoch` checks if there is a pending commission rate for the current epoch. If there is one, it updates the pool's commission rate to the new value. The function then splits the rewards between the pool's rewards and the operator's commission. This is done by utilizing the updated commission rate.

However, the updated commission rate is intended for future epochs and not for the current epoch. Thus, it will result in an incorrect calculation of the operator's commission.

Remediation

Ensure that the commission calculation utilizes the old commission rate for the current epoch.

Patch

Fixed in [cc4aaaa](#).

Bypassing Stake Threshold Check LOW

OS-MSW-ADV-09

Description

`ActiveSet::update` does not enforce a minimum stake threshold (`set.threshold_stake`), unlike `insert`. This introduces a vulnerability where a user may manipulate their stake to gain a position in the active set below the required threshold.

```
>_ sources/staking/active_set.move
```

RUST

```
/// Updates the staked amount of the storage node with the given `node_id` in
/// the active set. Returns true if the node is in the set.
public(package) fun update(set: &mut ActiveSet, node_id: ID, staked_amount: u64): bool {
    let index = set.nodes.find_index!(|entry| entry.node_id == node_id);
    if (index.is_none()) {
        return false
    };
    index.do!(|idx| {
        set.total_stake = set.total_stake + staked_amount - set.nodes[idx].staked_amount;
        set.nodes[idx].staked_amount = staked_amount;
    });
    true
}
```

For example, a user may stake an amount greater than `set.threshold_stake`, ensuring that their node is successfully inserted via `ActiveSet::insert`. If the active set is full, the node with the lowest stake is removed to accommodate the new entry. After successfully inserting, the user immediately calls `ActiveSet::update` to lower their stake below `set.threshold_stake`. As a result, the attacker secures a place in the active set without the required stake amount.

Remediation

Modify `update` to check if the new `staked_amount` meets the `threshold_stake` requirement before updating the node. Also, ensure `ActiveSet` removes nodes whose staked amount falls below the required `threshold_stake`.

Patch

Fixed in [134fee8](#).

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-MSW-SUG-00	<code>insert_or_update</code> updates the active set only when a stake or unstake action occurs. If a node leaves and another node qualifies but does not trigger a stake change, that node will not be added to the active set.
OS-MSW-SUG-01	There are several instances where proper validation is not performed, resulting in potential underflow issues.
OS-MSW-SUG-02	Recommendation for modifying the codebase for improved functionality, efficiency, and robustness.
OS-MSW-SUG-03	Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.

Failure to Replace Inactive Node

OS-MSW-SUG-00

Description

The issue with `insert_or_update` lies in its management of the `active_set` when nodes leave. Nodes may only join the `active_set` through direct staking actions (`stake` / `unstake`), rather than automatically filling vacant slots. As a result, if a node unstakes and exits the `active_set`, a replacement node with a sufficient `staked_amount` will not be added unless a `stake` or `unstake` action explicitly triggers `active_set::insert_or_update`.

```
>_ sources/staking/active_set.move
```

RUST

```
public(package) fun insert_or_update(set: &mut ActiveSet, node_id: ID, staked_amount: u64): bool
    ↪ {
    if (set.update(node_id, staked_amount)) {
        return true
    };
    set.insert(node_id, staked_amount)
}
```

Additionally, in the updated implementation, `insert_or_update` may implicitly remove an entry (by calling `set.remove(node_id)` if the `staked_amount` is invalid). Thus, it would be appropriate to properly handle the return value.

```
>_ sources/staking/active_set.move
```

RUST

```
public(package) fun insert_or_update(set: &mut ActiveSet, node_id: ID, staked_amount: u64): bool
    ↪ {
    // Currently, the `threshold_stake` is set to `0`, so we need to account for that.
    if (staked_amount == 0 || staked_amount < set.threshold_stake) {
        set.remove(node_id);
        return false
    };
    if (set.update(node_id, staked_amount)) true
    else set.insert(node_id, staked_amount)
}
```

Remediation

Add a method that lets nodes proactively attempt to join when an opening exists.

Patch

Fixed in [bb9e9e6](#).

Missing Validation Logic

OS-MSW-SUG-01

Description

1. In `system_state_inner::advance_epoch`, when calculating the storage contribution for the current `node_id`, the `deny_list_size` is subtracted from the total utilized capacity of the previous epoch (`old_epoch_used_capacity`). However, if the `deny_list_size` is greater than `old_epoch_used_capacity`, it will result in an overflow. Check if `old_epoch_used_capacity > deny_list_size` in order to prevent underflow.

```
>_ sources/system/system_state_inner.move
```

RUST

```
public(package) fun advance_epoch([...]): VecMap<ID, Balance<WAL>> {  
    [...]  
    node_ids.zip_do!(weights, |node_id, weight| {  
        let deny_list_size = deny_list_sizes.try_get(&node_id).destroy_or!(0);  
        let stored = (weight as u128) * ((old_epoch_used_capacity - deny_list_size) as  
            ↳ u128);  
        total_stored = total_stored + stored;  
        stored_vec.push_back(stored);  
    });  
    [...]  
}
```

2. `pending_values::reduce` should validate `value` is not greater than the current pending value for the given epoch to prevent an underflow.

```
>_ sources/staking/pending_values.move
```

RUST

```
public(package) fun reduce(self: &mut PendingValues, epoch: u32, value: u64) {  
    let map = &mut self.0;  
    if (!map.contains(&epoch)) {  
        abort EMissingEpochValue  
    } else {  
        let curr = map[&epoch];  
        *&mut map[&epoch] = curr - value;  
    };  
}
```

Remediation

Add the underflow checks as stated above.

Patch

1. Issue #1 fixed in [cc4aaaa](#).
2. Issue #2 fixed in [cc4aaaa](#).

Code Refactoring

OS-MSW-SUG-02

Description

1. `staking_pool::destroy_empty` checks `pending_stakes` twice, once by calling `pool.is_empty()`, and again at the end of the function. Ensure to remove the redundant check for `pending_stakes`.

```
>_ sources/system/staking_pool.move RUST  
  
/// Destroy the pool if it is empty.  
public(package) fun destroy_empty(pool: StakingPool) {  
    assert!(pool.is_empty(), EPoolNotEmpty);  
    [...]  
    let (_epochs, pending_stakes) = pending_stake.unwrap().into_keys_values();  
    pending_stakes.do!{|stake| assert!(stake == 0)};  
}
```

2. The current implementation of `staking::withdraw_node` will always abort, as both functions call `set_withdrawing`. Add a check to see if the node is already in the withdrawing state.
3. Currently, if `EpochState` is `NextParamsSelected`, deposits and withdrawals will be activated in `current_epoch + 2`. Therefore, `set_next_commission` should be triggered in `current_epoch + 3` to allow all users to react.

Remediation

Incorporate the above refactors into the codebase.

Patch

1. Issue #1 fixed in [cc4aaaa](#).
2. Issue #2 fixed in [cc4aaaa](#).

Code Maturity

OS-MSW-SUG-03

Description

1. The comment for `auth::authorized_object` is incorrect, as it states that `authorized_object` returns an address, whereas it actually returns an object. Update the comment to: *Returns the 'Authorized' as an object.*

```
>_ sources/staking/active_set.move
```

RUST

```
/// Returns the `Authorized` as an address.  
public fun authorized_object(id: ID): Authorized {  
    Authorized::ObjectID(id)  
}
```

2. The current implementation of `take_threshold_value` does not adhere to the functionality of `staking_inner::quorum_below` as described in the functions comments, as it returns the value that satisfies the quorum condition (`sum_weight >= threshold_weight`) without considering whether a slightly lower value may also have been supported by the quorum. If strictly following the comment, `quorum_below` should find the value just before `sum_weight >= threshold_weight` is satisfied in `take_threshold_value`.

```
>_ sources/staking/staking_inner.move
```

RUST

```
/// Take the lowest value, s.t. a quorum (2f + 1) voted for a value lower or equal to  
↪ this.  
fun quorum_below(vote_queue: &mut PriorityQueue<u64>, n_shards: u16): u64 {  
    let threshold_weight = ((n_shards - 1) / 3 + 1) as u64;  
    take_threshold_value(vote_queue, threshold_weight)  
}  
  
fun take_threshold_value(vote_queue: &mut PriorityQueue<u64>, threshold_weight: u64): u64  
↪ {  
    let mut sum_weight = 0;  
    // The loop will always succeed if `threshold_weight` is smaller than the total  
    ↪ weight.  
    loop {  
        let (value, weight) = vote_queue.pop_max();  
        sum_weight = sum_weight + weight;  
        if (sum_weight >= threshold_weight) {  
            return value  
        };  
    }  
}
```


3. Utilize the `used_capacity_size` of the new epoch in `system_state_inner::advance_epoch` instead of `old_epoch_used_capacity`, as this does not account for changes that may have occurred exactly at the epoch transition boundary.

Remediation

Implement the above-mentioned suggestions.

Patch

1. Issue #1 fixed in [422f76d](#).
2. Issue #2 fixed in [a1e81c6](#).
3. Issue #3 fixed in [a1e81c6](#).

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the [General Findings](#).

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
 - Improperly designed economic incentives leading to loss of funds.
-

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
 - Exploitation involving high capital requirement with respect to payout.
-

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
 - Forced exceptions in the normal user flow.
-

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.
-

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
 - Improved input validation.
-

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.